

9. Function & Class Templates

9.a Write a C++ Program to demonstrate generic programming using a function template

Aim:

To demonstrate generic programming using a function template.

Algorithm:

1. Start the program.
2. Declare a function template with type T.
3. Swap two variables of any type.
4. Call the template with integers.
5. Display the swapped values.
6. End the program.

Program:

```
#include <iostream>
using namespace std;
template <class T>
void swapValues(T &a, T &b)
{
    T temp = a;
    a = b;
    b = temp;
}

int main()
{
    int x = 5, y = 10;
    swapValues(x, y);
    cout << "x = " << x << ", y = " << y;
    return 0;
}
```

Output:

x = 10, y = 5

Result

Thus the C++ program has been executed successfully to demonstrate generic programming using a function template

9.b. Write a C++ Program to create a generic class using templates

Aim:

To create a generic class using templates.

Algorithm:

1. Start the program.
2. Declare a class template with type T.
3. Store a value of any type.
4. Create object with int or float.
5. Display stored value.
6. End the program.

Program:

```
#include <iostream>
using namespace std;
template <class T>
class Sample
{
    T data;
public:
    Sample(T d) { data = d; }
    void display() { cout << "Data = " << data; }
};

int main()
{
    Sample<int> obj(100);
    obj.display();
    return 0;
}
```

Output:

Data = 100

Result

Thus the C++ program has been executed successfully to create a generic class using templates